

ADR — Microserviço e Microfrontend de Autenticação

Plus — Sistema de Gestão de Estoque para Loja de Roupas Plus Size

Projeto	Sistema de Gestão de Estoque para Loja de Roupas Plus Size
Disciplina	Engenharia de Software II — 98802-02
Turma	30 — PUCRS / 2026-1
Professor	Prof. José Pedro Schardosim Simão
Entrega	T1 — Microserviço de Autenticação e Microfrontend de Autenticação
Versão	v1.0

1. Contexto e Problema

O sistema Plus utiliza arquitetura baseada em microserviços e microfrontends. Para centralizar autenticação e autorização dos usuários, foi definido um microserviço responsável pelo gerenciamento de identidade, autenticação e controle de acesso.

2. Decisão Arquitetural

A solução será composta por microserviço Node.js + Express, microfrontend React, PostgreSQL, autenticação JWT stateless e RBAC baseado em roles presentes no token JWT.

3. Tecnologias Adotadas

Backend: Node.js, Express, PostgreSQL, JWT, bcryptjs, Swagger/OpenAPI, Jest e Docker.
Frontend: React, Typescript, Material UI e Module Federation. Infraestrutura: Docker Compose, Terraform, Ministack e GitHub Actions.

4. Estrutura da Solução

Microserviço responsável por login, emissão de tokens JWT, middlewares de autenticação e RBAC, Swagger e testes automatizados. Microfrontend responsável pela autenticação, persistência local da sessão, interface construída com React + TypeScript e MUI e integração com o Shell App.

5. Modelo de Autenticação e Autorização

A autenticação será realizada via JWT. O frontend armazenará Access Token e Refresh Token em localStorage. O RBAC utilizará as roles ADMIN e VENDEDOR através de middlewares Express.

6. Persistência de Dados

O PostgreSQL armazenará usuários e informações de autenticação. A estrutura do banco será criada através de migrations SQL versionadas.

7. Segurança

As senhas serão protegidas utilizando bcryptjs. O logout removerá os tokens armazenados localmente no frontend.

8. Comunicação entre Serviços

O microfrontend realizará chamadas REST diretamente para o microserviço de autenticação.

9. Execução Local

A solução será executada via Docker Compose integrado ao plus-infra.

10. CI/CD

GitHub Actions automatizará instalação de dependências, execução de testes automatizados e validação de build TypeScript do frontend.

11. Swagger/OpenAPI

A API disponibilizará documentação Swagger/OpenAPI contendo endpoints, payloads, respostas e autenticação JWT.

12. Alternativas Consideradas

NestJS, sessão stateful, banco NoSQL e ACL completa foram considerados inadequados ao escopo simplificado do T1.

13. Trade-offs

A arquitetura privilegia modularidade e desacoplamento, porém adiciona complexidade operacional devido ao uso de múltiplos containers e gerenciamento de tokens JWT. Necessidade de configuração adicional de tipagem no frontend.

14. Consequências da Decisão

A adoção de React, Express e JWT facilita integração com arquitetura distribuída e padronização do ambiente utilizando Docker e GitHub Actions. O uso de TypeScript no frontend melhora padronização, manutenção e reduz erros relacionados à tipagem.